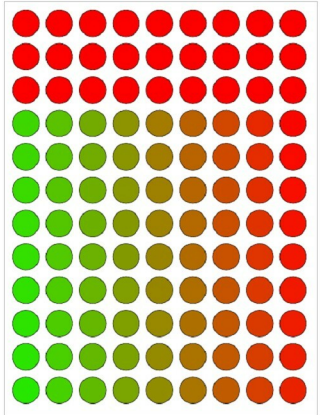
Figure 4: Output of *pgf6.tex*

(4mm); inside the inner loop gets executed for a total of 27 times and, therefore, draws 27 circles that each have a radius of 4mm. The centres of these circles are defined by the code fragment `(\x \y)`. Therefore, the centres are not fixed – they are the points (1,1), (1,2), (1,3), (2,1), (2,2), (2,3), ..., (9,1), (9,2) and (9,3), a total of 27 points in the two-dimensional plane as the values of variables *x* and *y* change inside the outer and inner loops.

The code fragment `[fill=red!\x!\y!green]` defines the colour filled inside the circles. The code fragment `red!40!green` defines a circle that has 40 per cent red and the remaining 60 per cent green. So, in this case, since the values of the variables *x* and *y* change inside the loops, no two circles will have the same colour. But it may not be possible to identify the difference in colour between a circle having 11 per cent red and 89 per cent green, and another circle with 12 per cent red and 88 per cent green. That is the reason why some of the adjacent circles may look similar in colour. On executing the command `pdflatex pgf6.tex` the LaTeX file *pgf6.tex* will produce an output file titled *pgf6.pdf* with the image shown in Figure 4.

Let us change just a single line of code in the file *pgf6.tex* to obtain *pgf7.tex*. The line of code `\foreach \y in {1,2,3} {` in *pgf6.tex* is replaced with the line `\foreach \y in {1,2,...,12} {` in *pgf7.tex*. On executing the file *pgf7.tex* with the command `pdflatex pgf7.tex, you will get an output file titled pgf7.pdf with the image shown in Figure 5. A few characters are added to a single line of code and the image changes drastically. I believe this example clearly illustrates the benefit of using graphical programming tools like PGF/TikZ and PSTricks rather than conventional image design software. Imagine how much time it would take to draw a technical diagram similar to the one we have discussed, with a lot of image components placed at specific locations, if we had used typical image design software.`

Now it is time to wind up our discussion. You should now be able to write and execute LaTeX scripts embedded with PGF/TikZ scripts to produce a DVI or a PDF file as output, either by using LaTeX or pdfLaTeX in the terminal or by using the popular LaTeX editor Texmaker. We have left out a few important topics like three-dimensional plotting, plotting in the beamer document class of LaTeX, etc, but the introduction given in this

Figure 5: Output of *pgf7.tex*

article will definitely provide you with a head start over others. In my opinion, PGF/TikZ is a powerful weapon in the hands of professionals who need to produce good quality technical diagrams.

Why learn PSTricks?

PSTricks is yet another powerful scientific graphics visualisation tool used along with LaTeX. One last question I want to answer before concluding this article is, 'Why learn PSTricks if PGF/TikZ is so great?' Well, PGF/TikZ is definitely a great tool but PSTricks also has something good to offer. As mentioned earlier, PSTricks is associated with PostScript, a very powerful page description language. The programming power offered by PostScript makes it absolutely essential that I should also introduce PSTricks, so that potential users can experiment with both PGF/TikZ as well as PSTricks, and then make an informed decision regarding their choice for a good visualisation tool for LaTeX. So, in the next and final part of this series on scientific graphics visualisation, we will discuss PSTricks. 

By: Deepu Benson

The author is a free software enthusiast and his area of interest is theoretical computer science. He maintains a technical blog at www.computingforbeginners.blogspot.in and can be reached at deepumb@hotmail.com.